# Precision Support for Row-Major × Row-Major GPU Matrix Multiplication: FP16, BF16, and FP32 on Tensor Cores

ganesh@cs.utah.edu
(written with Claude Code)

September 25, 2026

**Abstract**

This report studies matrix multiplication $C = A \times B$ where both $A$ and $B$ arrive in **row-major** layout, restricted to three precisions: FP16, BF16, and FP32. Integer (INT8/IMMA) matrix multiplication is out of scope and has been commented out of `tc_matmult/leimao-mm.cu` accordingly. Every claim about what the hardware and CUDA actually support here is verified directly against the CUDA 12.4 `mma.hpp` header shipped on this machine (path in Section 2), not general recollection — exact fragment types, tile shapes, and accumulator types are quoted from it. FP16 is fully exercised, working code (`leimao-mm.cu`); BF16 and the FP32/TF32 path are supported by the same header but not exercised by that file. Section 5 answers directly: no SIMT-core (ordinary per-thread CUDA) matrix multiplication happens anywhere in this file — every GPU-side multiply-accumulate goes through tensor cores.

## Contents

**1 Scope and What "Row-Major, Row-Major" Means Here** **2**

**2 FP16 (HMMA): Working, and Already in the File** **2**
2.1 What the header actually defines . . . . . 2
2.2 How the multiplication is realized . . . . . 2

**3 BF16: Supported by the Header, Not Exercised by This File** **3**

**4 FP32: Not on Tensor Cores at Full Precision — Only via TF32** **3**
4.1 What the header does *not* define . . . . . 4
4.2 What the header does define: TF32 . . . . . 4
4.3 If true full-precision FP32 GEMM is actually required . . . . . 4

**5 Is Any of This SIMT-Core Matrix Multiplication?** **5**

**6 SIMT vs. Tensor Core: A Real Benchmark (FP16)** **5**

**7 Summary Table** **6**

**8 Conclusion** **6**

# 1 Scope and What "Row-Major, Row-Major" Means Here

`leimao-mm.cu` (Lei Mao's WMMA GEMM example, copied verbatim into this repository) exposes its GEMM through a column-major-plus-transpose-flag interface:

```
launch_wmma_mm(A, B, C, m, n, k, is_A_transpose, is_B_transpose, stream);
```

Its own header comment states the convention precisely: *"All the data in the matrices are stored in a column-major order… WMMA_FRAG_LAYOUT_A: row_major if A is transposed, otherwise col_major."* This is a standard trick, not a real transpose: an $M \times K$ matrix stored **row-major** occupies *exactly* the same bytes in memory as its $K \times M$ transpose stored **column-major**. So "`is_A_transpose=true`" in this code does not mean "compute $A^T$" in any mathematical sense that changes the answer — it means "the data `A` points to is laid out row-major; load it that way." The code confirms this directly: setting the flag selects `nvcuda::wmma::row_major` as the fragment load layout, and adjusts the leading dimension accordingly (`lda = is_A_transpose ? k : m`, correct for a row-major $M \times K$ array).

**Consequence for this study:** row-major $A$ and row-major $B$ is obtained with

```
launch_wmma_mm(A, B, C, m, n, k, /*is_A_transpose=*/true, /*is_B_transpose=*/true,
    stream);
```

— the branch the demo's own `print_test_header` labels "`C = A^T * B^T`". That label is accurate to the code's internal naming convention but *not* a mathematical transpose of the matrices this report cares about: it is exactly $C = A \times B$ with both operands read as ordinary row-major arrays.

# 2 FP16 (HMMA): Working, and Already in the File

## 2.1 What the header actually defines

From `crt/mma.hpp`, CUDA 12.4 (full path on this machine: `/usr/local/cuda-12.4/targets/x86_64-linux/include/crt/mma.hpp`): `wmma::fragment<matrix_a/matrix_b, 16, 16, 16, __half, row_major|col_major>` is defined, with `load_matrix_sync` for both layouts. Two families of `mma_sync` overloads exist for the accumulator: one into `fragment<accumulator, 16, 16, 16, __half>` (FP16 accumulation) and one into `fragment<accumulator, 16, 16, 16, float>` (FP32 accumulation) — *both* are real, hardware-backed instruction sequences, not a software emulation of one from the other. The same three families of tile shapes appear for FP16 as for every other supported type here: $16 \times 16 \times 16$, $32 \times 8 \times 16$, and $8 \times 32 \times 16$.

`leimao-mm.cu` always instantiates `launch_wmma_mm<__half, float>` — i.e. it always chooses the **FP32-accumulator** HMMA path, at the $16 \times 16 \times 16$ tile shape, and never exercises the FP16-accumulator variant even though the hardware/header support it.

## 2.2 How the multiplication is realized

One CUDA *warp* (32 threads acting as one unit) computes one $16 \times 16$ output tile per iteration of the outer loop, striding over $K$ in steps of 16:

```
warpM, warpN <- this warp's 2D tile coordinates (from blockIdx/threadIdx)
acc_frag: fragment<accumulator, 16,16,16, float> <- fill_fragment(0)
for ki = 0, 16, 32, ..., k-16:
```

```
    a_frag: fragment<matrix_a, 16,16,16, half, row_major>
    b_frag: fragment<matrix_b, 16,16,16, half, row_major>
    load_matrix_sync(a_frag, A + offset(warpM, ki), lda) # row-major load
    load_matrix_sync(b_frag, B + offset(ki, warpN), ldb) # row-major load
    mma_sync(acc_frag, a_frag, b_frag, acc_frag) # one HMMA tensor-core op
c_frag: fragment<accumulator, 16,16,16, float> <- load existing C tile
for each element e in c_frag: c_frag[e] = alpha*acc_frag[e] + beta*c_frag[e]
store_matrix_sync(C + offset(warpM, warpN), c_frag, ldc, mem_col_major)
```

Every multiply-accumulate over the $16 \times 16 \times 16$ tile happens in a single `mma_sync` call — one tensor-core HMMA instruction, not 4096 separate multiply-adds issued by ordinary CUDA cores. `load_matrix_sync` and `store_matrix_sync` handle the row-major↔fragment layout translation entirely inside the intrinsic; the kernel code never manually transposes anything.

## 3 BF16: Supported by the Header, Not Exercised by This File

**Yes, BF16 is supported** — on Ampere (compute capability 8.0+, which includes `sm_86`, the target this file already builds for) and confirmed directly in the same header:

```
wmma::fragment<matrix_a/matrix_b, 16, 16, 16, __nv_bfloat16, row_major|col_major>
```

is defined, at all three tile shapes (16×16×16, 32×8×16, 8×32×16), with full `load_matrix_sync` support for both layouts.

**One structural difference from FP16, confirmed by the header:** only `mma_sync` overloads accumulating into `fragment<accumulator, 16,16,16, float>` exist for BF16 — there is *no* BF16-accumulator overload. BF16 tensor-core matrix multiplication on this hardware **always** accumulates in FP32; there is no choice to make, unlike FP16.

`leimao-mm.cu` never instantiates a BF16 path — `__nv_bfloat16` does not appear anywhere in the file. Adding it would be a direct, mechanical change: swap `__half` for `__nv_bfloat16` as `T1` (keeping `T2 = float`, since that is the only accumulator type available), and replace `float2half`'s `__float2half` conversion with `__float2bfloat16`. The kernel body is otherwise identical to Section 2:

```
acc_frag: fragment<accumulator, 16,16,16, float> <- fill_fragment(0) # float is
    mandatory
for ki = 0, 16, 32, ..., k-16:
    a_frag: fragment<matrix_a, 16,16,16, __nv_bfloat16, row_major>
    b_frag: fragment<matrix_b, 16,16,16, __nv_bfloat16, row_major>
    load_matrix_sync(a_frag, A + offset(warpM, ki), lda)
    load_matrix_sync(b_frag, B + offset(ki, warpN), ldb)
    mma_sync(acc_frag, a_frag, b_frag, acc_frag) # one BF16 tensor-core op
# store identical to FP16 case, c_frag is float throughout
```

## 4 FP32: Not on Tensor Cores at Full Precision — Only via TF32

This is the precision where the answer is not "yes, same pattern, different type."

### 4.1 What the header does *not* define

Searching `crt/mma.hpp` for a tensor-core fragment holding true, full-mantissa `float` operands (23-bit mantissa, IEEE-754 single precision) for `matrix_a`/`matrix_b` finds nothing. **Tensor cores on this GPU cannot multiply two full-precision FP32 matrices directly.** There is no WMMA instruction for it, at any tile shape.

### 4.2 What the header does define: TF32

`wmma::fragment<matrix_a/matrix_b, 16, 16, 8, precision::tf32, row_major|col_major>` — note the $K$ dimension is **8**, not 16, unlike every other type in this report. `load_matrix_sync` for this fragment still takes a plain `const float*` pointer: the caller's data in host/device memory stays ordinary FP32 the whole time. The precision reduction — TF32 keeps FP32's 8-bit exponent but truncates the mantissa to 10 bits, matching FP16's mantissa width while keeping FP32's exponent range — happens *inside* the load intrinsic itself, not as a separate conversion step the caller writes. The accumulator, `fragment<accumulator, 16,16,8, float>`, is full-precision `float`, and `mma_sync` overloads for all four row/col-major combinations of `matrix_a`/`matrix_b` are present, confirming TF32 GEMM is fully supported for row-major operands, exactly like Sections 2 and 3.

`leimao-mm.cu` does not exercise this path either — no `precision::tf32` anywhere in the file. A TF32 version of the kernel:

```
acc_frag: fragment<accumulator, 16,16,8, float> <- fill_fragment(0)
for ki = 0, 8, 16, ..., k-8: # K-tile is 8, not 16
    a_frag: fragment<matrix_a, 16,16,8, precision::tf32, row_major>
    b_frag: fragment<matrix_b, 16,16,8, precision::tf32, row_major>
    load_matrix_sync(a_frag, A_float_ptr + offset(warpM, ki), lda) # FP32 in, TF32
        internally
    load_matrix_sync(b_frag, B_float_ptr + offset(ki, warpN), ldb)
    mma_sync(acc_frag, a_frag, b_frag, acc_frag) # one TF32 tensor-core op
# store: c_frag is float, full precision, no further loss on the output/accumulate side
```

### 4.3 If true full-precision FP32 GEMM is actually required

Then it cannot run on tensor cores on this hardware at all — it has to be an ordinary SIMT kernel using plain `FFMA` instructions on the regular CUDA cores, ordinary per-thread arithmetic, no `wmma` namespace involved:

```
# one thread per output element (a real kernel would tile through shared memory;
# this is the conceptual/unoptimized form)
row, col <- this thread's output coordinates
acc: float = 0
for ki = 0, 1, 2, ..., k-1:
    acc += A[row, ki] * B[ki, col] # ordinary FFMA, full 23-bit mantissa throughout
C[row, col] = acc
```

`leimao-mm.cu` does not contain this kernel either (see Section 5) — its only full-precision FP32 code is the CPU-side reference function, not a GPU kernel at all.

## 5 Is Any of This SIMT-Core Matrix Multiplication?

No. `leimao-mm.cu` contains exactly one `__global__` kernel, `wmma_gemm_a_col_major_b_col_major`, and every multiply-accumulate inside it happens via `nvcuda::wmma::mma_sync` — a tensor-core instruction. There is no kernel anywhere in this file that computes a matrix product using ordinary per-thread CUDA arithmetic (`FFMA`, `HFMA2`, or otherwise) on the GPU.

The one non-tensor-core matrix multiplication in the file, `mm_a_col_major_b_col_major`, is **not a GPU kernel at all**: it has no `__global__` or `__device__` qualifier, takes no thread indices, and is called directly from `main()` on the host. It is a plain, serial, triple-nested-loop C++ function that runs on the **CPU**, used purely to compute a correctness reference (`get_avg_abs_diff_ratio` compares it against the tensor-core result) — its runtime is never measured or reported, unlike the tensor-core kernel's (`measure_performance`, cudaEvent-timed).

**Summary: 100% of GPU-side matrix multiplication in `leimao-mm.cu` is tensor-core work. 0% is SIMT-core.** A true SIMT GEMM kernel, as sketched in Section 4, does not exist in `leimao-mm.cu` in any form — it has now been written from scratch as a separate pair of files, `leimao-simt-variant-fp32.cu` and `leimao-simt-variant-fp16.cu`, benchmarked directly against the tensor-core path in Section 6.

## 6 SIMT vs. Tensor Core: A Real Benchmark (FP16)

`leimao-simt-variant-fp32.cu` and `leimao-simt-variant-fp16.cu` are the SIMT kernels Section 5 found missing from `leimao-mm.cu` — a standard shared-memory tiled GEMM ($32 \times 32$ tiles, one thread per output element, ordinary `FFMA` throughout, ordinary CUDA (SIMT) cores, no `wmma` namespace anywhere), restricted to exactly the row-major × row-major case this report studies (Section 1). The FP16 variant matches `leimao-mm.cu`'s own accumulation choice for a fair comparison: `__half` inputs, `float` accumulation (via `__half2float` inside the inner loop), `float` output — the same precision contract as `launch_wmma_mm<__half, float>`, so the only thing actually being compared is *which hardware path performs the multiply-accumulate*, not a difference in numerical precision along the way. Both SIMT files reuse the exact same `measure_performance` (cudaEvent-timed, 10 warmups, 10 measured repeats) and $1024 \times 1024 \times 1024$ problem size as `leimao-mm.cu`, and both were checked for correctness against a CPU reference before timing (no "high average absolute diff ratio" warning printed by either — i.e. within the same 0.01 average relative-error tolerance `leimao-mm.cu` itself uses).

| | Latency (ms) | |
|---|---|---|
| Tensor core (HMMA, `leimao-mm.cu`, "C = A^T * B^T") | 0.177–0.178 | (3 runs) |
| SIMT (`leimao-simt-variant-fp16.cu`) | 0.864–0.866 | (3 runs) |

Averaging each: tensor core ≈ 0.178 ms, SIMT ≈ 0.865 ms — **the tensor-core path is ≈4.9× faster** than this tiled SIMT kernel, same GPU, same problem size, same input/accumulation precision, same $1024^3$-element problem. This is a real, measured number from this machine, not a textbook or vendor figure; it should not be read as "the" tensor-core speedup in general — a more aggressively tuned SIMT kernel (larger tiles, register blocking, double-buffered shared memory, vectorized loads) would close some of this gap, and the gap itself grows or shrinks with problem size and GPU generation. What it does establish concretely: even a reasonably standard, textbook-tiled SIMT GEMM is not close to tensor-core throughput on this hardware for this problem size

— the $16 \times 16 \times 16$-element HMMA instruction Section 2 walks through is doing, per issue, what this SIMT kernel needs $16^3 = 4096$ separate scalar `FFMA`-equivalent operations to accomplish.

## 7 Summary Table

| Precision | Tensor core? | Fragment type | Accumulator | Tile ($M \times N \times K$) |
|---|---|---|---|---|
| FP16 | Yes (HMMA) | `__half` | `__half` *or* `float` | 16×16×16 (+2 more) |
| BF16 | Yes | `__nv_bfloat16` | `float` only | 16×16×16 (+2 more) |
| FP32 (TF32) | Yes | `precision::tf32` | `float` | 16×16×**8** |
| FP32 (full) | No | — (SIMT only) | `float` | N/A |

| Precision | Exercised in `tc_matmult/`? |
|---|---|
| FP16, tensor core | Yes — `leimao-mm.cu`'s only working GEMM path (FP32 accumulator) |
| FP16, SIMT | Yes — `leimao-simt-variant-fp16.cu` (Section 6) |
| FP32, SIMT | Yes — `leimao-simt-variant-fp32.cu` |
| BF16 | No — supported by the header/hardware, absent from this directory |
| FP32 (TF32) | No — supported by the header/hardware, absent from this directory |

**A note beyond the three precisions in scope.** The same header also defines `fragment<matrix_a/matrix_b, 8, 8, 4, double, row_major|col_major>` — double-precision (DMMA) tensor-core matrix multiplication exists too, at a much smaller 8×8×4 tile. Out of scope for this report, but worth knowing it isn't only FP16/BF16/TF32/INT8 that get tensor-core paths on this architecture family.

## 8 Conclusion

For row-major $A$ times row-major $B$ (Section 1): FP16 works today, unmodified, via `leimao-mm.cu`'s existing `is_A_transpose=true, is_B_transpose=true` call. BF16 is fully supported by the same CUDA/hardware stack, confirmed directly in `mma.hpp`, but needs new code — a direct, same-shape port of the FP16 path with the element type swapped and the accumulator forced to `float`. FP32 has no true full-precision tensor-core path at all; the tensor-core option is TF32 (an explicit, hardware-managed precision reduction on load, full-precision accumulate, smaller $K$-tile), and genuine full-precision FP32 GEMM would require an entirely separate, non-tensor-core SIMT kernel. Every GPU-side multiply-accumulate that exists in `leimao-mm.cu` itself runs on tensor cores; its only non-tensor-core matrix multiply is a CPU-side correctness check, not a GPU kernel.

That missing SIMT kernel now exists, in both precisions (Section 6), and the FP16 comparison in Section 6 puts a real number on the gap this report otherwise only described structurally: on this GPU, at $1024 \times 1024 \times 1024$, with matched input/accumulation precision, tensor cores finish in $\approx 0.178$ ms against a reasonably standard tiled SIMT kernel's $\approx 0.865$ ms — roughly 4.9×. Section 2's structural point (one HMMA instruction does the work of 4096 scalar multiply-adds) and this measured 4.9× are two different kinds of evidence for the same conclusion, not the same number restated: the instruction count says why a gap should exist; the benchmark says how large it actually is for this kernel, this problem size, this GPU.